CLASS 01

YOUR JOURNEY INTO WEB DEVELOPMENT

Orientation & Web Foundations

How the web works • Client & server • HTTP • Developer setup • HTML

A practical first step — no prior experience required.

01 Understand

Explore what happens behind a website.

02 Set up

Get your developer tools ready.

03 Build

Create your first HTML webpage.

What we'll learn today

01	**Web basics**	What web development is and where websites live.
02	**How the web works**	URLs, DNS, clients, servers and HTTP.
03	**Developer setup**	VS Code, terminal, Node.js and npm.
04	**HTML foundations**	Document structure, tags and attributes.
05	**Build & practise**	Create a first webpage and test your knowledge.

What is web development?

Web development is the process of creating, building and maintaining websites and web applications that run on the internet or an internal network.

Frontend

The part people see and interact with.

Examples: buttons, forms, navigation, layouts.

HTML • CSS • JavaScript • React

Backend

The logic working behind the scenes.

Examples: login, processing requests, business rules.

Node.js • Express • Python • Java

Database

Stores and organises application data.

Examples: user profiles, orders, product details.

MongoDB • MySQL • PostgreSQL

A typical full-stack application brings these parts together.

Website or web application?

Website

- Often focuses on sharing information.
- May have limited interaction.
- Content is the main purpose.

Example: an informational organisation website.

Web application

- Focuses on user tasks and interaction.
- Often uses dynamic data.
- Users sign in, create, edit or manage content.

Examples: Gmail, Google Docs.

The boundary is not strict — modern websites often include web-app features.

How does the web work?

1 Enter URL

You type a website address.

2 Find server

DNS looks up the domain's IP address.

3 Connect

Browser establishes a network connection.

4 Request

Browser sends an HTTP request.

5 Respond

Server processes and returns resources.

6 Render

Browser builds and displays the page.

Example: when you visit google.com, the browser has to locate the server, communicate with it and load the resources.

What is a URL?

URL = Uniform Resource Locator

```
https://www.example.com:443/products?id=101
```

https	www	example.com	:443	/products	?id=101
Protocol	Subdomain	Domain	Port	Path	Query

Protocol defines how communication happens • Domain is the readable address • Path identifies a resource • Query parameters pass extra information.

What are a client and a server?

CLIENT

- A device or application requesting a resource.
- Usually a browser or mobile app.
- Sends requests to a server.

Example: Chrome on your laptop.

REQUEST →

← RESPONSE

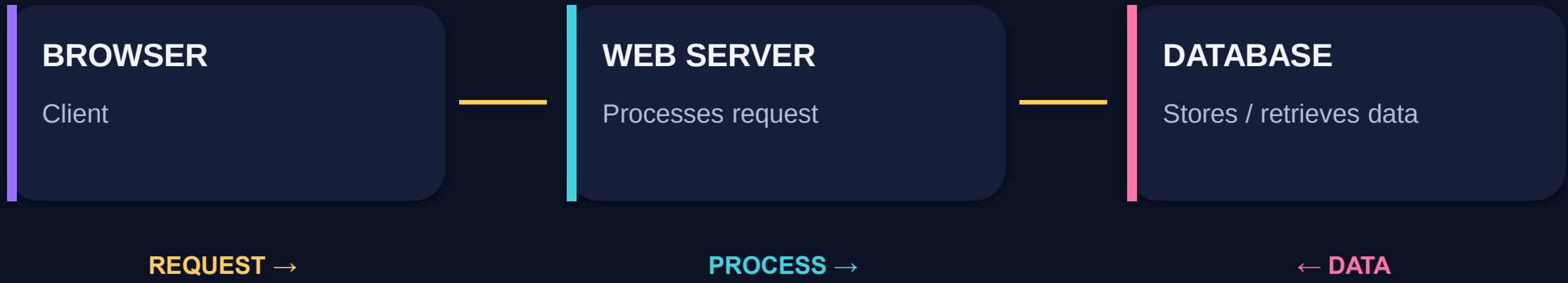
SERVER

- A computer or software system that receives requests.
- Processes the request.
- Sends a response to the client.

Example: a website's backend service.

Login example: your browser sends login details; the backend checks them and returns a result.

How client and server communicate



The server sends an HTTP response back to the browser. The browser then renders the result for the user.

The database is not always involved — simple static files can be served without a database query.

What is HTTP?

HTTP = HyperText Transfer Protocol

HTTP is an application-layer protocol used by clients and servers to exchange web resources and data.

Why do we need it?

- Request webpages and resources
- Send information to servers
- Retrieve data from APIs
- Connect frontend and backend

What about HTTPS?

HTTPS is HTTP protected with TLS.

TLS helps encrypt communication and authenticate the server, protecting data in transit.

Use HTTPS for real websites and never send sensitive information over plain HTTP.

Inside an HTTP request and response

REQUEST

Method • URL • Headers • Optional body

```
POST /api/users HTTP/1.1
Host: example.com
Content-Type: application/json

{"name":"Bhavika","email":"bhavika@example.com"}
```

RESPONSE

Status code • Headers • Optional body

```
HTTP/1.1 201 Created
Content-Type: application/json

{"message":"User created successfully"}
```

Example only: a request body can contain submitted data; a response body can contain returned data.

HTTP methods

GET	**Retrieve data**	Read a product list
POST	**Submit / create**	Register a new user
PUT	**Replace a resource**	Replace profile details
PATCH	**Partially update**	Change a username
DELETE	**Remove a resource**	Delete an account

Memory trick: GET = read • POST = create • PUT = replace • PATCH = partial update • DELETE = remove

HTTP status codes

2xx	**Success**	200 OK • 201 Created • 204 No Content
3xx	**Redirection**	301 Moved Permanently • 302 Found • 304 Not Modified
4xx	**Client error**	400 Bad Request • 401 Unauthorized • 403 Forbidden • 404 Not Found
5xx	**Server error**	500 Internal Server Error • 502 Bad Gateway • 503 Service Unavailable

A 404 usually means the requested resource was not found — it does not automatically mean the entire website is down.

What does a web developer need?

VS Code

Write and edit code

Browser

Run and test webpages

Terminal

Execute commands

Node.js

Run JavaScript outside browser

npm

Install and manage packages

Git & GitHub

Track changes and collaborate

We'll use these tools throughout the web development journey.

Meet Visual Studio Code

Visual Studio Code (VS Code) is a free source-code editor that helps developers write, edit, debug and manage code.

Syntax highlighting

Makes code easier to scan.

IntelliSense

Offers code suggestions and autocompletion.

Extensions

Adds useful tools and features.

Integrated terminal

Run commands without leaving the editor.

Debugging

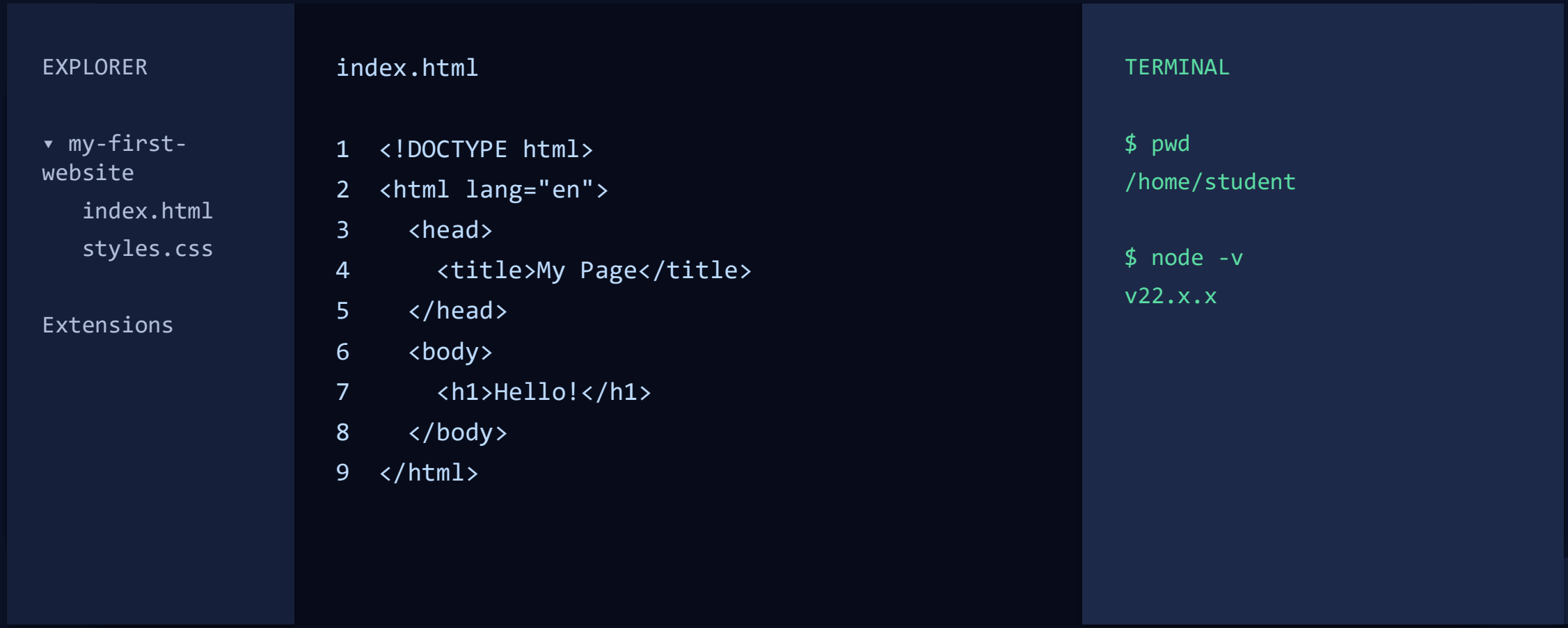
Helps find and fix problems.

File Explorer

Organise project folders and files.

Download: code.visualstudio.com

Explore the VS Code interface



What is HTML?

HTML = HyperText Markup Language

HTML is the standard markup language used to structure content on web pages.

HTML

Structure

Headings, paragraphs, links, images, forms.

CSS

Presentation

Colours, spacing, layout, typography, responsive design.

JavaScript

Behaviour

Events, dynamic content, interactivity, logic.

Think of a house: HTML = structure • CSS = interior design • JavaScript = switches and functionality.

Reference: developer.mozilla.org/en-US/docs/Web/HTML

HTML elements, tags and attributes

```
<h1>Hello World</h1>
```

Break it down

<h1> opening tag • Hello World content • </h1> closing tag

Common HTML elements

`<h1> – <h6>`	Headings	`<p>`	Paragraph
`<a>`	Hyperlink	`<img>`	Image
`<div> / <span>`	Containers	`<ul> / <ol> / <li>`	Lists
`<button>`	Button		

What are attributes?

Attributes provide extra information about an element.

Visit Google

<img src="photo.jpg" alt="My photo">

href = link destination • src = image source • alt = alternative text

Note: <img> is a void element — it does not wrap content or need a closing tag.

The structure of an HTML document

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
  <title>My First Website</title>
</head>
<body>
  <h1>Hello World!</h1>
  <p>Welcome to my first website.</p>
</body>
</html>
```

`<!DOCTYPE html>`	Declares an HTML5 document.
`<html>`	Root element; lang sets document language.
`<head>`	Metadata and resource information.
`meta charset`	Sets character encoding.
`viewport`	Helps the page adapt to screen sizes.
`<title>`	Text shown in the browser tab.
`<body>`	Main content displayed on the page.

Head = information about the page • Body = content users see

Let's build our first webpage!

YOUR FIRST HTML FILE

1. Open VS Code
2. Create a folder: my-first-website
3. Create a file: index.html
4. Add the HTML boilerplate
5. Save and open it in a browser

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>My Introduction</title>
</head>
<body>
  <h1>Hello, I'm Bhavika!</h1>
  <p>I'm learning web development.</p>
</body>
</html>
```

Open index.html directly in your browser, or use Live Server to preview it locally.

What have we learned today?

Web development

Building websites and web applications

Frontend

The user-facing interface

Backend

Server-side logic and processing

Client / server

Requests and responses

HTTP / HTTPS

Web communication / protected with TLS

VS Code / terminal

Editor / command-line interface

Node.js / npm

JavaScript runtime / package manager

HTML

Structure of a webpage

After-class challenge

Build a personal webpage

Include:

- Your name as the main heading
- A profile picture
- A short introduction
- A list of hobbies or interests
- A clickable favourite website link
- At least three heading levels
- A meaningful page title

Submission & bonus

Bonus: Add a section called “My Web Development Goals” and write three goals.

Submission: Share your project folder or GitHub repository on Google Classroom.

Keep the code readable and test the page in a browser.

Small steps, consistent practice. Your first webpage is only the beginning.

CLASS 01 COMPLETE

YOU DON'T BECOME A DEVELOPER BY JUST WATCHING TUTORIALS.

You become one by building, experimenting, breaking things, and learning how to fix them.

Keep Learning. Keep Building. Keep Growing.
Always cheering for you,
Your Mentor — Bhavika ♥